

Coding project

We are going to read some seminal papers in information security this semester. One of which is the short essay "[Reflections on Trusting Trust](#)" by Ken Thompson. Without spoiling it too much, Thompson describes how a compiler can be rigged to insert backdoors into other programs--essentially bugging the program without needing to bug the program's source code. Of course, that same concept can be applied to the compiler itself, yielding a compiler that is rigged to bug another program without any evidence of the bug in either the compiler's or the target program's source code.

Your task is to implement this attack on a C compiler of your choosing.

With that said, I highly recommend you not try to insert your backdoor into gcc or llvm--they take a very long time to build. Instead, try the Tiny C Compiler which you can find on GitHub here: <https://github.com/TinyCC/tinycc>. This C compiler can rebuild itself in under a second, so you can very quickly test changes. There is a tcc binary and a clean copy of the source code on the shell server.

You may work in groups (of up to 4) for this assignment. However, each of you individually must be able to explain how your exploit works. To that end, each individual must submit a write-up (1-2 pages) on how the exploit works. All code should live on GitHub. Make a new repo (can be private) and invite me to be a collaborator. My GitHub username is @ibara.

Deliverables:

The finished bugged compiler, a quality git log of the commits to the compiler and login.c so I can see the stages of development, any modifications you made during development to the login.c source code that the compiler inserts the backdoor into, and the write-up. All the code can live in the same GitHub repo. Email me your individual write-up.

Review procedure:

I will take your bugged compiler source code from GitHub and run it through the tcc compiler on the shell server to produce Compiler A. I will then run the clean tcc source code through Compiler A to produce Compiler B. I will use Compiler B to compile the original login.c and then run the produced login binary (so please tell me what the other account name that returns 0 is in your write-up).

You should follow this procedure to ensure things are working correctly before submission.

Grading breakdown:

Write-up: 5 points

Everything else: 20 points

Due on the last day of class, 4/23 EOD.

Here is the program (login.c) you will insert your backdoor into:

```
#include <string.h>

/*
 * Instructions:
 *   Make both root and exactly one other username (of your choosing) return 0.
 *   You may edit this file however you'd like while developing your exploit.
 *   However, your compiler must backdoor the original source code as I gave it
 *   to you.
 *   There may not be any evidence of the backdoor in this or the compiler's
 *   source code.
 *
 * Notes:
 *   You can check the return code in the shell by running `echo $?`.
 *
 * Compiling:
 *   cc -o login login.c
 */

static int
do_login(const char *username)
{
    if (!strcmp(username, "root"))
        return 0;

    return 1;
}

int
main(int argc, char *argv[])
{
    if (argc != 2)
        return 1;

    return do_login(argv[1]);
}
```